10 Real-Life Critical Thinking Problems

Applying Weeks 1–5 Knowledge to Real-World Scenarios

Instructions for Students:
For each problem below, you must:

1. **Analyze** the scenario and identify what data you need to store
2. **Choose** appropriate data types and structures (array vs ArrayList)
3. **Design** the logic using conditionals and loops
4. **Write** the Java program with proper comments
5. **Test** with sample inputs and edge cases

These problems require you to combine multiple concepts from Weeks 1–5. There is no single "right" answer – focus on your reasoning and problem-solving approach.

Problem 1: M-Pesa Agent Float Management

Scenario:
You are developing a simple system for an M-Pesa agent in Kawangware. The agent starts each day with a float (cash reserve) of KSh 50,000. Throughout the day, customers come to:

- **Deposit money** (agent receives cash, sends e-money) → float increases
- **Withdraw money** (agent gives cash, receives e-money) → float decreases

The agent wants to know:

- The float balance after each transaction
- How many deposits and withdrawals were made
- Whether the agent needs to rebalance (if float drops below KSh 10,000 or exceeds KSh 100,000)

Your Task:
Write a Java program that:

1. Reads the number of transactions to process
2. For each transaction, reads the type (1 = deposit, 2 = withdrawal) and amount
3. Updates the float balance accordingly

4. Displays a warning if the float goes outside the safe range
5. After all transactions, displays a summary (total deposits, total withdrawals, final balance)

Concepts to Apply:

- Scanner for input (Week 2)
- if-else / switch for transaction type (Week 3)
- for loop for processing transactions (Week 4)
- Variables and operators for balance calculation (Week 2)
- Any other topics/concepts that are relevant in solving the problem as you have already learned from week 1 through week 5

Critical Thinking Questions:

- What happens if a withdrawal exceeds the available float?
- How would you store the transaction history for later review?
- What if the agent wants to process transactions until they type "done"?

Problem 2: Student Fee Payment Tracker

Scenario:
A small college in Nakuru has 8 students in a class. Each student is required to pay KSh 45,000 per semester. The students make payments at different times. The administrator wants a program that can:

- Record payments for each student
- Track the balance for each student
- Identify students who have fully paid, partially paid, or not paid at all

Your Task:
Write a Java program that:

1. Stores the names of 8 students in an array
2. Creates an array to store the total amount paid by each student
3. Repeatedly asks the administrator to enter a student index (0–7) and a payment amount

4. Updates the student's payment record
5. When the administrator enters -1, the program stops and displays:
    - Each student's name, amount paid, and balance
    - A list of students who have fully paid
    - A list of students with outstanding balances

Concepts to Apply:

- Arrays for storing fixed data (Week 5)
- while or do-while loop for repeated input (Week 4)
- if-else for categorization (Week 3)
- Scanner for input (Week 2)
- Any other topics/concepts that are relevant in solving the problem as you have already learned from week 1 through week 5

Critical Thinking Questions:

- What happens if the administrator enters an invalid student index?
- How would you prevent overpayment (paying more than the required amount)?
- Could you use an ArrayList instead of an array? What are the trade-offs?

Problem 3: Traffic Light Simulation for a Busy Nairobi Junction

Scenario:
The Nairobi City Council wants to simulate a traffic light at a busy junction. The light cycles through:

- **Green** for 30 seconds
- **Yellow** for 5 seconds
- **Red** for 25 seconds

During rush hour, the green light stays on for 45 seconds instead of 30. The system should also count how many complete cycles occur in a given time period.

Your Task:
Write a Java program that:

1. Asks the user if it is rush hour (yes/no)
2. Asks for the total simulation time in seconds
3. Simulates the traffic light by printing the color and remaining seconds at each second
4. Counts and displays the number of complete cycles

Concepts to Apply:

- Scanner for input (Week 2)
- boolean for rush hour flag (Week 2)
- for or while loop for simulation (Week 4)
- if-else for determining light color (Week 3)
- Modulus operator for cycling (Week 2)
- Any other topics/concepts that are relevant in solving the problem as you have already learned from week 1 through week 5

Critical Thinking Questions:

- How would you handle a simulation time that doesn't complete a full cycle?
- What if the user wants to change the timings mid-simulation?
- How would you store the sequence of colors for analysis?

Problem 4: Supermarket Barcode Checker

Scenario:
A supermarket in Mombasa uses barcodes to identify products. Each barcode is a 13-digit number. The last digit is a **check digit** computed from the first 12 digits. The formula is:

1. Sum the digits in odd positions (1st, 3rd, 5th, ..., 11th) → sumOdd
2. Sum the digits in even positions (2nd, 4th, 6th, ..., 12th) → sumEven
3. Calculate: checkDigit = (10 - ((sumOdd + 3 * sumEven) % 10)) % 10

Your Task:
Write a Java program that:

1. Reads a 13-digit barcode as a String
2. Extracts each digit using charAt() and converts to int
3. Calculates the check digit
4. Compares it with the last digit of the barcode
5. Displays "Valid Barcode" or "Invalid Barcode"

Concepts to Apply:

- String and charAt() (Week 2)
- Type conversion (char to int) (Week 2)
- Loops for processing digits (Week 4)
- if-else for validation (Week 3)
- Operators for calculation (Week 2)
- Any other topics/concepts that are relevant in solving the problem as you have already learned from week 1 through week 5

Critical Thinking Questions:

- What happens if the user enters a barcode that isn't 13 digits?
- How would you validate that all characters are digits?
- Can you adapt this to a 12-digit UPC barcode?

Problem 5: Hospital Patient Temperature Monitoring

Scenario:
A hospital in Kisumu monitors patient temperatures every hour for 24 hours. Normal temperature is between 36.1°C and 37.2°C. A temperature above 37.5°C indicates fever; below 36.0°C indicates hypothermia.

Your Task:
Write a Java program that:

1. Creates an array of 24 double values (one for each hour)

2. Reads the temperatures from the user
3. Analyzes the data and displays:
    - The highest and lowest temperatures
    - The average temperature
    - The hours when the patient had a fever (above 37.5°C)
    - The hours when the patient had hypothermia (below 36.0°C)
    - Whether the patient's temperature was stable (all readings within 0.5°C of each other)

Concepts to Apply:

- Arrays for storing hourly readings (Week 5)
- Loops for reading and analyzing data (Week 4)
- if-else for categorization (Week 3)
- Operators for calculations (Week 2)
- Any other topics/concepts that are relevant in solving the problem as you have already learned from week 1 through week 5

Critical Thinking Questions:

- How would you calculate "stability"?
- What if the patient was discharged early (fewer than 24 readings)?
- How would you visualize this data (without a GUI)?

Problem 6: Boda Boda Ride Fare Calculator

Scenario:
A boda boda rider in Eldoret charges fares based on distance and time of day:

- **Base fare:** KSh 50 for the first 2 km
- **Additional distance:** KSh 20 per km after the first 2 km
- **Night surcharge (10 PM – 6 AM):** 50% extra
- **Rush hour (7 AM – 9 AM, 5 PM – 7 PM):** 25% extra

- **Rain:** KSh 30 flat extra

Your Task:
Write a Java program that:

1. Reads the distance (in km), hour of day (0–23), and whether it's raining (yes/no)
2. Calculates the fare using the rules above
3. Displays the breakdown (base fare, distance charge, surcharges, total)
4. Processes multiple rides until the rider enters distance 0

Concepts to Apply:

- Scanner for input (Week 2)
- if-else-if for surcharge determination (Week 3)
- while or do-while loop for multiple rides (Week 4)
- Operators for fare calculation (Week 2)
- Any other topics/concepts that are relevant in solving the problem as you have already learned from week 1 through week 5

Critical Thinking Questions:

- What if the distance is negative?
- How would you round the fare to the nearest 10 shillings?
- How would you track total earnings for the day?

Problem 7: Class Attendance and Exam Eligibility System

Scenario:
A university requires students to have at least 75% attendance to sit for final exams. A class has 10 students, and there are 20 lectures in the semester. Each student's attendance is recorded as 1 (present) or 0 (absent) for each lecture.

Your Task:
Write a Java program that:

1. Creates a 2D array int[10][20] to store attendance (10 students × 20 lectures)
2. Fills the array with random 0s and 1s (or reads from user)

3. For each student, calculates attendance percentage
4. Displays a table showing:
    - Student ID, lectures attended, percentage, eligibility status
5. Counts and displays the total number of eligible and ineligible students

Concepts to Apply:

- 2D arrays (Week 5)
- Nested loops for processing 2D data (Week 4)
- if-else for eligibility (Week 3)
- Operators for percentage calculation (Week 2)
- Any other topics/concepts that are relevant in solving the problem as you have already learned from week 1 through week 5

Critical Thinking Questions:

- How would you identify the lecture with the lowest attendance?
- What if a student joins mid-semester (fewer than 20 lectures)?
- How would you handle a student who is sick and has a medical certificate?

Problem 8: Airtime Reseller Inventory System

Scenario:
A small airtime reseller in Thika buys airtime in bulk and sells it to customers. The reseller tracks the following denominations: 10, 20, 50, 100, 250, and 500 shillings.

Your Task:
Write a Java program that:

1. Creates two parallel arrays: one for denominations (int[]) and one for quantities (int[])
2. Repeatedly asks the user to:
    - Sell airtime (enter denomination and quantity)
    - Restock airtime (enter denomination and quantity)
    - View current inventory

    - Exit
3. Updates the inventory after each transaction
4. Displays a warning if any denomination falls below 5 units
5. On exit, displays:
    - Current inventory levels
    - Total value of remaining stock

Concepts to Apply:

- Parallel arrays (Week 5)
- do-while loop for menu (Week 4)
- switch for menu options (Week 3)
- if-else for low-stock warnings (Week 3)
- Operators for value calculation (Week 2)
- Any other topics/concepts that are relevant in solving the problem as you have already learned from week 1 through week 5

Critical Thinking Questions:

- What happens if the user tries to sell more than available?
- How would you add a new denomination to the system?
- How would you track sales history for the day?

Problem 9: Water Bill Calculator for Nairobi Residents

Scenario:
Nairobi Water charges residents based on a tiered system:

- **First 6 m^3:** KSh 50 per m^3
- **Next 10 m^3 (7–16):** KSh 80 per m^3
- **Next 20 m^3 (17–36):** KSh 120 per m^3
- **Above 36 m^3:** KSh 150 per m^3
- **Fixed service charge:** KSh 200

- **Sewerage charge:** 75% of water charge

Your Task:
Write a Java program that:

1. Reads the water consumption in m^3
2. Calculates the water charge using the tiered system
3. Adds the service charge and sewerage charge
4. Displays a detailed bill with a breakdown
5. Processes multiple customers until the user enters -1

Concepts to Apply:

- while loop for multiple customers (Week 4)
- if-else-if for tiered calculation (Week 3)
- Operators for arithmetic (Week 2)
- Scanner for input (Week 2)
- Any other topics/concepts that are relevant in solving the problem as you have already learned from week 1 through week 5

Critical Thinking Questions:

- How would you validate that consumption is not negative?
- How would you format the bill to look professional?
- What if you wanted to store all customer bills for the month?

Problem 10: Simple Library Book Management System

Scenario:
A small community library in Nyeri has a collection of books. Each book has a title, author, and availability status (available or borrowed). The librarian wants a simple system to manage the collection.

Your Task:
Write a Java program that:

1. Uses an ArrayList<String> for book titles and an ArrayList<Boolean> for availability

2. Provides a menu with options:
    - Add a book
    - Borrow a book (by title)
    - Return a book (by title)
    - Display all books with status
    - Search for a book by title
    - Exit
3. Updates availability when books are borrowed or returned
4. Prevents borrowing a book that is already borrowed
5. Displays the total number of books, available books, and borrowed books

Concepts to Apply:

- ArrayList for dynamic book collection (Week 5)
- do-while loop for menu (Week 4)
- switch for menu options (Week 3)
- if-else for availability logic (Week 3)
- String methods for search (Week 2)
- Any other topics/concepts that are relevant in solving the problem as you have already learned from week 1 through week 5

Critical Thinking Questions:

- How would you handle multiple copies of the same book?
- How would you track who borrowed which book?
- How would you add a "due date" and calculate fines for late returns?

✓ **Concept Coverage Matrix**

Problem	Variables	Operators	Conditionals	Loops	Arrays	ArrayList
1. M-Pesa Float	✅	✅	✅	✅		
2. Fee Tracker	✅	✅	✅	✅	✅	
3. Traffic Light	✅	✅	✅	✅		
4. Barcode Checker	✅	✅	✅	✅		
5. Temperature Monitor	✅	✅	✅	✅	✅	
6. Boda Boda Fare	✅	✅	✅	✅		
7. Attendance System	✅	✅	✅	✅	✅ (2D)	
8. Airtime Inventory	✅	✅	✅	✅	✅	
9. Water Bill	✅	✅	✅	✅		

Problem	Variables	Operators	Conditionals	Loops	Arrays	ArrayList
10. Library System	✅	✅	✅	✅		✅

✓ **How to Approach These Problems**

Step 1: Understand the Problem

- Read the scenario carefully.
- Identify inputs, processes, and outputs.
- Ask: "What data do I need to store?"

Step 2: Plan Your Data Structures

- What variables do I need?
- Should I use an array or an ArrayList?
- Do I need a 2D array?

Step 3: Design the Logic

- What conditionals do I need?
- What loops do I need?
- What operations do I need to perform?

Step 4: Write the Code

- Start with the skeleton (class, main method, Scanner).
- Add variable declarations.
- Add the logic step by step.
- Add comments explaining each part.

Step 5: Test and Refine

- Test with normal inputs.

- Test with edge cases (zero, negative, large values).
- Test with invalid inputs.

✓ **Extension Challenges**

For students who finish early, note the following:

1. **I insist please** Save data to a file and read it back. This will be used to preview of Week 11.
2. **For validation purpose:** Ensure all user inputs are valid before processing.
3. **Reporting:** Generate a summary report at the end of the program.
4. **Refactor into methods:** Break your code into reusable methods this will be used as a (preview of Week 6).

END OF CRITICAL THINKING PROBLEMS

"The best way to learn to solve problems is to solve problems." – Anonymous

---

Instructor Note: These problems are designed to be challenging but achievable with Weeks 1–5 knowledge. I encourage you to attempt the problems. The goal is not just to get the "right answer" but to develop a systematic approach to problem-solving.